

CODSOFT Artificial Intelligence Internship

SMARTFLIX RECOMMENDATION SYSTEM

> **SUBMITTED BY:** - [Khushi Maina](#)

> **INTERNSHIP DOMAIN:** - [Artificial Intelligence](#)

> **ORGANIZATION:** - [CODSOFT](#)

Introduction

Recommendation systems are widely used in modern applications such as movie streaming platforms, e-commerce websites, and social media applications. They help users find relevant content based on their preferences.

The SmartFlix Recommendation System is a console-based Python application that recommends movies according to the genre selected by the user. The system stores movies in different categories and randomly suggests movies from the chosen genre.

Objective

The objective of this project is to develop a simple movie recommendation system that suggests movies to users based on their favorite genre. The system helps users discover movies according to their interests and demonstrates the implementation of recommendation concepts using Python.

Software Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 2 GB RAM
- Keyboard and Monitor

Software Requirements

- Python 3.x
- Visual Studio Code / IDLE / PyCharm

- Windows, Linux, or MacOS

Functional Requirements

1. Display available movie genres.
2. Accept genre input from the user.
3. Validate the entered genre.
4. Recommend movies from the selected category.
5. Generate recommendations randomly.
6. Allow users to request additional recommendations.
7. Exit the system when the user chooses.

System Design

Input

- User's favorite movie genre

Processing

- Match the genre with stored movie categories.
- Randomly select movies from the category.

Output

- Display recommended movies.

Algorithm

Step 1

Start the program.

Step 2

Display available movie genres.

Step 3

Accept genre input from the user.

Step 4

Convert input into lowercase format.

Step 5

Check whether the genre exists in the movie database.

Step 6

If genre exists:

- Display recommendation message.
- Select random movies using the random module.
- Display recommendations.

Step 7

If genre does not exist:

- Display an error message.

Step 8

Ask the user whether they want another recommendation.

Step 9

If user enters "y", repeat the process.

Step 10

Otherwise terminate the program.

Modules Used

Random Module

The random module is used to generate random movie recommendations.

Example:

```
recommendations = random.sample(  
    movies[genre],  
    min(3, len(movies[genre]))  
)
```

Data Structure Used

Dictionary

The movie database is stored using a Python dictionary.

Example:

```
movies = {  
    "action": [  
        "John Wick",  
        "Mad Max: Fury Road",  
        "The Dark Knight"  
    ]  
}
```

Advantages:

- Fast searching
- Easy category management
- Organized data storage

Functions and Logic

Genre Selection

The user selects a movie genre from the available list.

Recommendation Engine

The system checks the selected genre and retrieves movies from the corresponding category.

Random Recommendation

Movies are selected randomly to provide different recommendations each time.

Loop Control

The program continues until the user chooses to exit.

Source Code

```
import random
```

```
print("=" * 50)
print("SMARTFLIX RECOMMENDATION")
print("=" * 50)
```

```
movies = {
    "action": [
        "John Wick",
        "Mad Max: Fury Road",
        "The Dark Knight"
    ],

    "comedy": [
        "3 Idiots",
        "Hera Pheri",
        "jumanji:Welocome to the jungle"
    ],

    "science fiction": [
        "Interstellar",
        "Avatar",
        "The Martian"
    ],

    "horror": [
        "The Conjuring",
```

```
"It",
"Annabelle"
],

"romantic": [
    "Titanic",
    "La La Land",
    "Me before you"
]
}

while True:

    print("\nAvailable genres:")
    print("1. Action")
    print("2. Comedy")
    print("3. Science Fiction")
    print("4. Horror")
    print("5. Romantic")

    genre = input("\nEnter your favorite genre: ").lower()

    if genre in movies:

        print("\nAnalyzing your preferences...")
```

```
print("\nRecommended movies for you:")

recommendations = random.sample(
    movies[genre],
    min(3, len(movies[genre]))
)

for i, movie in enumerate(recommendations, start=1):
    print(f"{i}. {movie}")

else:
    print("\nGenre not found. Please try again.")

choice = input("\nDo you want another recommendation? (y/n): ")

if choice.lower() != "y":
    print("\nThank you for using Movie Recommendation System!")
    breaky
```

Sample Output

Figure: - Menu Module

```
=====
SMARTFLIX RECOMMENDATION
=====

Available genres:
1. Action
2. Comedy
3. Science Fiction
4. Horror
5. Romantic

Enter your favorite genre: █
```

Figure: - 2 Sample

```
Enter your favorite genre: Comedy

Analyzing your preferences...

Recommended movies for you:
1. jumanji: Welocome to the jungle
2. 3 Idiots
3. Hera Pheri

Do you want another recommendation? (y/n): █
```

Advantages

- Easy to use.
- Fast recommendations.
- Beginner-friendly project.
- Demonstrates Python dictionaries and lists.

- Uses random recommendation logic.
- Interactive user interface.

Limitations

- Small movie database.
- Recommendations are genre-based only.
- No user history tracking.
- No rating system.
- No machine learning implementation.

Future Enhancements

- Add a larger movie database.
- Include movie ratings and reviews.
- Implement machine learning algorithms.
- Add personalized recommendations.
- Develop a graphical user interface (GUI).
- Connect with online movie databases.
- Add search functionality.

Applications

- Movie recommendation platforms.
- Entertainment applications.
- Learning recommendation system concepts.
- Educational Python projects.
- Prototype for advanced recommendation engines.

Conclusion

The SmartFlix Recommendation System successfully recommends movies based on user-selected genres. The project demonstrates the use of Python dictionaries, lists, loops, conditional statements, and randomization techniques. It provides a simple yet effective

example of how recommendation systems work and serves as a foundation for developing more advanced intelligent recommendation engines in the future.